Automating Shellcode Patching with Python

Red Team Collection

Image: Persephone, the queen of the underworld

Preface/Introduction

The tools I engineered for this write-up are simple but effective implementations to automate patching of a sample 64-bit metasploit shellcode. Persephone, the Python tool that is the main focus of this write-up, can be found here. SharpLoader, the shellcode loader used to ensure functionality is maintained during testing, can be found here. This process has been great for my learning, I would encourage anyone to try and make their own implementations of these tools.

Reference the "Shellcode Patching v2" post for the manual process where we used Hades Scanner from 0xtriboulet and Binary Ninja to manually patch a sample shellcode, then used Hasherezade's loader to test for any functionality issues.

The example workflow used in the previous write-up boils down to:

- Scan a shellcode sample with Hades Scanner
- Use output from Hades to navigate to the offset where the flagged data is stored in Binary ninja
- Examine the opcodes in the disassembly for the flagged bytes in the Hades output
- Look for any modifiable instructions
- Make a modification
- Save the patched shellcode as a different named file
- Test to see if the functionality is in tact with a loader

In this write-up, we will simply pipe the output of Hades into a file and pass it as a parameter to Persephone along with the shellcode file scanned by Hades.

Thus, we have effectively automated the workflow. This is a simple foundation for more advanced applications in the future.

```
C:\Users\dco_admin\Desktop\implants\C Maldev>HadesV3.py --ruleset_path yara-rules-full.yar calc.bin > flagged_strings

C:\Users\dco_admin\Desktop\implants\C Maldev>type flagged_strings
Rule: ELASTIC_Windows_Trojan_Metasploit_7Bc0F998
        String match: $a1
        Offset: 0x00000011
        Data: 4831d265488b5260488b5218488b5220488b7250480fb74a4a4d31c94831c0ac3c61
Rule: ELASTIC_Windows_Trojan_Metasploit_C9773203
        String match: $a
        Offset: 0x00000075
        Data: 4831c0ac41c1c90d4101c138e075f14c034c24084539d1

C:\Users\dco_admin\Desktop\implants\C Maldev>Persephone.py flagged_strings calc.bin

Rule: ELASTIC_Windows_Trojan_Metasploit_7Bc0F998
[+] xor operation detected at offset 00000011, would you like to patch this operation? y/n
y
Patched 4831d2 to 31d290 at offset 00000011
[+] xor operation detected at offset 00000011, would you like to patch this operation? y/n
y
Failed to patch 4831c0 at offset 00000011

Rule: ELASTIC_Windows_Trojan_Metasploit_C9773203
[+] xor operation detected at offset 00000075, would you like to patch this operation? y/n
y
Patched 4831c0 to 31c090 at offset 00000075
Patched shellcode saved as zagreus.bin

C:\Users\dco_admin\Desktop\implants\C Maldev>HadesV3.py --ruleset_path yara-rules-full.yar zagreus.bin

C:\Users\dco_admin\Desktop\implants\C Maldev>runshc64.exe zagreus.bin
[*] Reading module from: zagreus.bin
>>> Creating a new thread...
[*] Running the shellcode [20c1d630000 - 20c1d630114]

C:\Users\dco_admin\Desktop\implants\C Maldev>
```

The first snippet:

```
>HadesV3.py --ruleset_path yara-rules-full.yar calc.bin > flagged_strings
```

We simply pipe the output of the Hades scan into a file called “flagged_strings”

```
C:\Users\[redacted]\Desktop\implants\C Maldev>type flagged_strings
Rule: ELASTIC_Windows_Trojan_Metasploit_7Bc0F998
        String match: $a1
        Offset: 0x00000011
        Data: 4831d265488b5260488b5218488b5220488b7250480fb74a4a4d31c94831c0ac3c61
Rule: ELASTIC_Windows_Trojan_Metasploit_C9773203
        String match: $a
        Offset: 0x00000075
        Data: 4831c0ac41c1c90d4101c138e075f14c034c24084539d1
```

In the above snippet, the contents of “flagged_strings” is printed to the screen

```
C:\Users\[redacted]\Desktop\implants\C Maldev>Persephone.py flagged_strings calc.bin
```

Next, we invoke Persephone and pass the flagged_strings file along with the shellcode file as parameters

```
Rule: ELASTIC_Windows_Trojan_Metasploit_7Bc0F998
[+] xor operation detected at offset 00000011, would you like to patch this operation? y/n
y
Patched 4831d2 to 31d290 at offset 00000011
[+] xor operation detected at offset 00000011, would you like to patch this operation? y/n
y
Failed to patch 4831c0 at offset 00000011

Rule: ELASTIC_Windows_Trojan_Metasploit_C9773203
[+] xor operation detected at offset 00000075, would you like to patch this operation? y/n
y
Patched 4831c0 to 31c090 at offset 00000075
Patched shellcode saved as zagreus.bin
```

Next, the script checks the string of flagged bytes for any simple changes it can make to patch it without affecting functionality. The only check that exists as of the initial release is for the xor operation being performed on a 64-bit general purpose register against itself:

0x00000011 4831d2 xor rdx,rdx

Persephone will drop the "48" opcode then append a null byte followed by the remainder of the string.

The following snippet highlights this functionality in the code itself:

```
xor_patterns = {
    '4831d2': '31d290',
    '4831c0': '31c090',
    '4831c9': '31c990',
    '4831db': '31db90',
    '4831e4': '31e490',
    '4831ed': '31ed90',
    '4831f6': '31f690',
    '4831ff': '31ff90',
}
```

The final key piece, Persephone outputs a file titled "zagreus.bin" in the directory from which it was run.

```
Patched shellcode saved as zagreus.bin
```

"Zagreus" was the child of Persephone and Hades. Nice little easter-egg lol

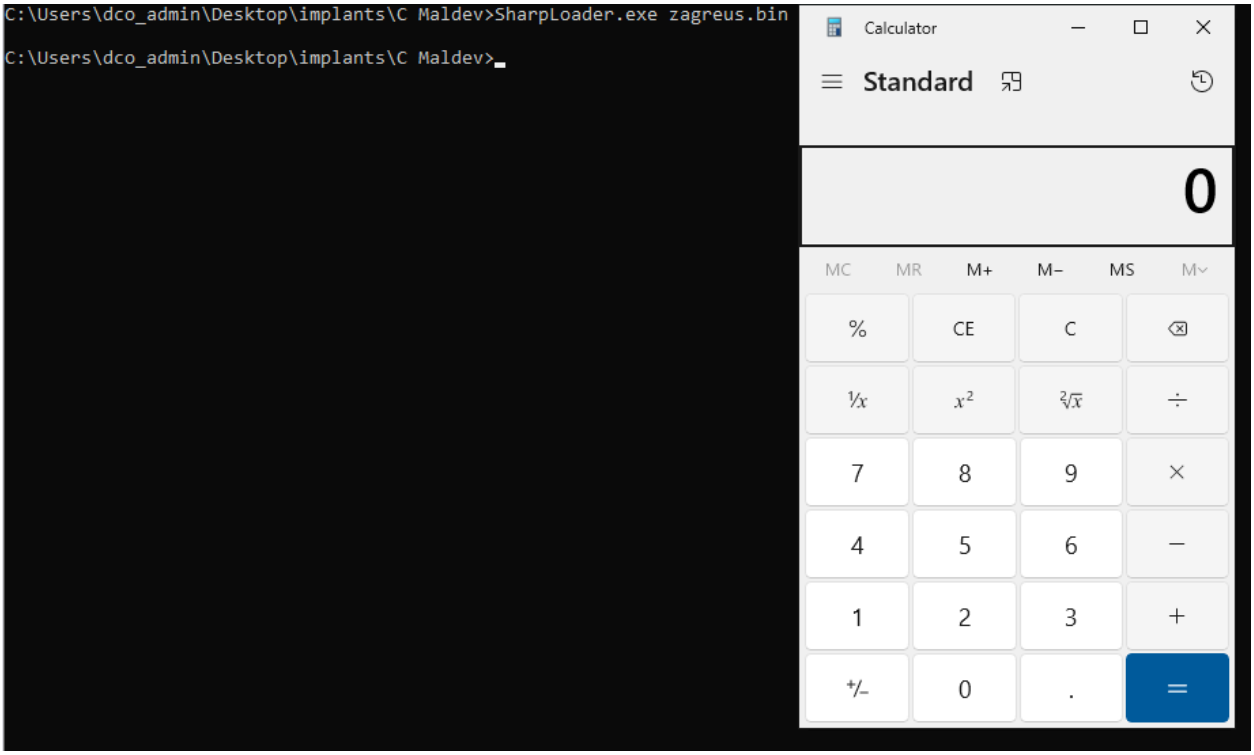
Now, let's test the outputted shellcode file to see if it now evades the alerts…

```
C:\Users\[redacted]\Desktop\implants\C Maldev>HadesV3.py --ruleset_path yara-rules-full.yar zagreus.bin

C:\Users\[redacted]\Desktop\implants\C Maldev>[redacted]
```

Yep!

Now let's do a functionality test:



Conclusion:
This was a really fun project, I hope you were able to learn something worthwhile from it. If you have any questions just let me know! I will be updating Persephone's checks as I continue to dive into assembly. Thanks for reading!

Tools:
https://github.com/Steveo21/Persephone
https://github.com/Steveo21/SharpLoader